

Predicting Hospital Readmission in Diabetes Patients

Nathan Gin, Alexander Yu, Nhan Nguyen
CS273A Machine Learning

Abstract

We looked at the 30-day hospital readmission prediction among diabetic patients using the Diabetes 130-US Hospitals dataset. The data presented noisy categorical variables, irregular missingness, and large variance amongst clinical profiles. After exploratory analysis and data preprocessing, we train and compare Logistic Regression, Random Forests, Support Vector Machines (SVMs), and Multilayer Perceptrons (MLPs). The focus of this report is not only on final performance but also on model selection, parameter choices, and diagnosing under-fitting versus over-fitting using cross-validation and complexity curves. On a split test set, a tuned Random Forest reaches ROC-AUC ≈ 0.69 and F1 ≈ 0.62 , with a tuned MLP close behind. We discuss how dataset characteristics motivated our feature design and how our tuning decisions were driven by validation performance.

1 Introduction

Unplanned hospital readmissions are costly and often indicate poor transitions from hospital care to home care. Patients with diabetes are high risk because they tend to have multiple medical conditions, medication, and visits with the healthcare system. Accurate prediction of readmission risk can help improve follow-up resources and improve patients lives.

The goal is to build and compare several machine learning models for predicting whether a hospitalized diabetic patient will be readmitted ****within 30 days**** after discharge. This task serves as a case study in feature design, model selection, and complexity control.

We first perform exploratory analysis to understand missingness patterns, outcome balance, and basic relationships between demographics and the other variables. We use these insights to guide preprocessing. Next, we explore techniques less focused on in class such as Random Forests, SVMs with RBF kernels, and Neural Networks where we tune their hyperparameters. After, we apply stratified cross-validation for parameter selection and inspect complexity curves to examine under-fitting and over-fitting. Finally, we evaluate all models on a separate test set and compare accuracy, precision, recall, F1, and ROC-AUC.

2 Data and Preprocessing

2.1 Dataset and target definition

The Diabetes 130-US Hospitals dataset contains 101,766 encounters and approximately 50 variables, including demographics, ICD-9 diagnosis codes, lab results, procedures, medication use, and admission/discharge information. The original outcome variable `readmitted` has three levels: “NO”, “< 30” (readmitted within 30 days), and “> 30” (readmitted at some later time).

For this project we define a binary target

$$y = \begin{cases} 1 & \text{if readmitted} < 30 \text{ or } > 30, \\ 0 & \text{if NO readmission is recorded.} \end{cases}$$

Under this definition the cleaned dataset has 99,340 encounters with about 52.9% non-readmitted and 47.1% readmitted, which is substantially more balanced than the original three-class formulation.

Figure 2.1 shows the raw distribution of readmission categories and how age bands relate to readmission.

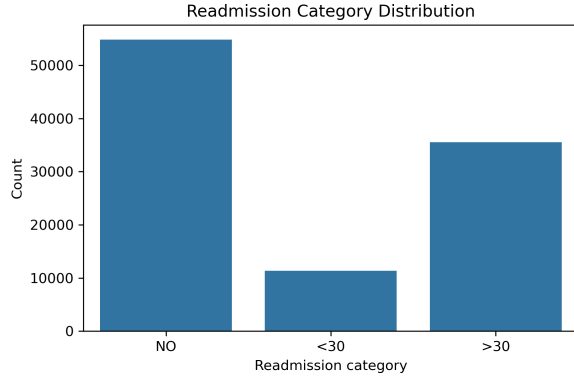


Figure 1: Raw readmission categories (NO, < 30, > 30).

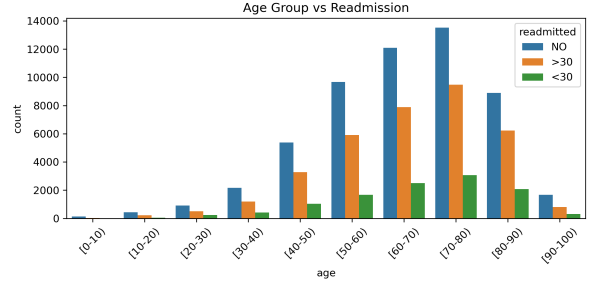


Figure 2: Age group vs. readmission, suggesting higher risk in older patients.

2.2 Missingness and feature design

Several variables use “?” as a missing-value code. Figure 2.2 (left) shows that **weight**, **payer_code**, and **medical_specialty** are missing in 40–98% of encounters. These columns were dropped because their sparsity made them difficult to model reliably. Diagnosis columns and key utilization variables had much lower missingness and were retained.

ICD-9 diagnosis codes are extremely high-dimensional and sparse. We mapped the three diagnosis fields (**diag_1--3**) into 8 broad groups (circulatory, respiratory, digestive, endocrine/diabetes, injury, musculoskeletal, genitourinary, neoplasms, and other) using simple ranges on the numeric codes. This grouping reduces dimensionality while preserving broad clinical patterns.

Twenty-one diabetes-related medications had four levels (“No”, “Steady”, “Up”, “Down”). We collapsed these into binary indicators for “any use or adjustment” versus “no use”. This decision was based on both sparsity and the observation that any non-zero level roughly indicates active management.

Hospital admission and discharge codes were grouped into a smaller set of clinically similar categories and then one-hot encoded. Gender, diabetes medication flag (**diabetesMed**), and medication change flag (**change**) were coded as 0/1. Age bands were mapped to integer codes 0–9.

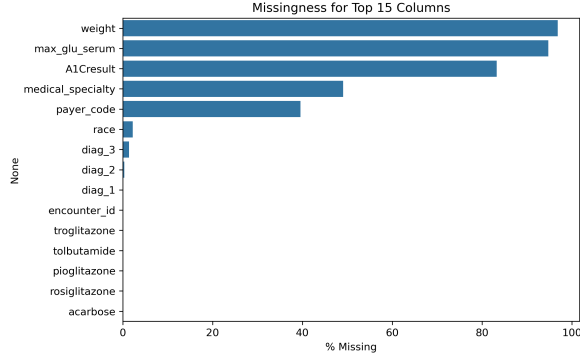


Figure 3: Missingness in the 15 most affected columns. Extremely sparse features were dropped.

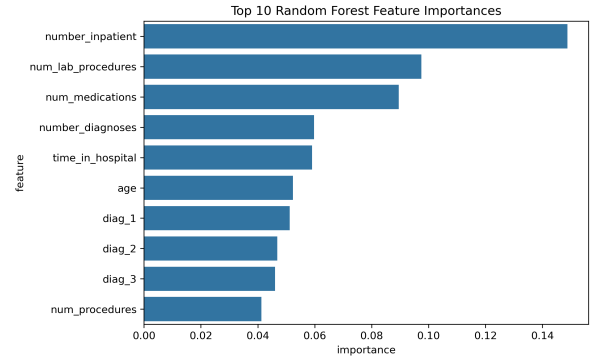


Figure 4: Top Random Forest feature importances, highlighting utilization and diagnosis variables.

After preprocessing, the final feature matrix contained 60 columns, including grouped diagnoses, one-hot encoded categorical variables, medication indicators, and utilization counts such as number of inpatient visits, procedures, and medications.

2.3 Train–test split and balancing

We used an 80/20 stratified split into train and test sets. To improve model training, we addressed class imbalance only in the training set: the majority class (not readmitted) was downsampled to match the minority class, producing a balanced training sample of 37,453 readmitted and 37,453 non-readmitted encounters. The test set retained the original 53/47 prevalence so that evaluation reflects a realistic clinical distribution.

A **StandardScaler** was fitted on the balanced training features and then applied to both training and test sets for models sensitive to feature scaling (Logistic Regression, SVM, and MLP). Tree-based models (Decision Trees and Random Forests) used the unscaled features.

3 Models and Hyperparameter Selection

We implemented four main model families: Logistic Regression, Random Forests, SVMs with RBF kernel, and MLP Neural Networks. Our emphasis was on how we chose hyperparameters and controlled complexity, rather than on exhaustive grid search.

Stratified k -fold cross-validation on the balanced training data was used throughout for model comparison and tuning, with ROC–AUC as the primary score.

3.1 Logistic Regression

Logistic Regression serves as a linear baseline and a sanity check for the more complex models. We used:

- Solver: **lbfgs**
- Class weights: **balanced**
- Maximum iterations: 300

This configuration requires little tuning and provides interpretable coefficients. It also gives a point of comparison for how much benefit the nonlinear models provide.

3.2 Random Forests

Random Forests are well-suited to this dataset because they:

- Handle a mix of continuous and categorical (one-hot) variables.
- Capture nonlinear interactions between features such as diagnoses, prior utilization, and medication use.
- Are relatively robust to noisy predictors and outliers.

Rather than running a large automatic grid search, we manually explored a small set of parameter combinations based on bias–variance considerations:

- `n_estimators` $\in \{100, 200\}$
- `max_depth` $\in \{10, 20, \text{None}\}$
- `min_samples_leaf` $\in \{1, 5\}$

For each configuration we computed 5-fold stratified CV ROC–AUC.

Depth values larger than 20 gave training accuracy close to 1.0 but did not improve validation AUC, indicating overfitting. Increasing `min_samples_leaf` from 1 to 5 slightly reduced training performance but improved validation performance and stabilized feature importance.

The best model according to cross-validation was:

$$\text{n_estimators} = 200, \quad \text{max_depth} = 20, \quad \text{min_samples_leaf} = 5,$$

with mean CV ROC–AUC ≈ 0.685 . We refer to this as the “tuned” Random Forest.

3.3 Support Vector Machine (RBF)

We considered SVMs with RBF kernel as a flexible, margin-based nonlinear model. However, training RBF SVMs on tens of thousands of samples is computationally expensive ($\mathcal{O}(n^2)$). To keep training feasible we:

- Subsampled 20,000 examples from the balanced training set.
- Used `C = 1.0` as a moderate regularization strength; higher C values led to increased variance without clear gains.
- Used `gamma = ‘scale’`, which adapts the kernel width to the feature dimensionality.
- Set `class_weight = ‘balanced’`.

Under these settings, SVM performance on the test set was comparable to Logistic Regression but did not surpass the tuned Random Forest or MLP.

3.4 Multilayer Perceptron (MLP)

The MLP is the most flexible model considered here and required careful complexity control. The dataset has only moderate signal-to-noise ratio and relatively simple structure, so very deep networks risk severe overfitting.

We varied three aspects of the architecture and optimization:

- Hidden layer structures: (32), (64), (128), and (128, 64).
- L2 regularization strength $\alpha \in \{10^{-4}, 10^{-3}, 10^{-2}\}$.
- Initial learning rate $\in \{10^{-3}, 5 \times 10^{-4}\}$.

Other settings were fixed: ReLU activations, batch size 256, maximum 50 epochs, and early stopping based on validation loss.

Figure 4.1 (right) shows a complexity curve where we vary only the hidden layer size and plot mean training and validation accuracy across CV folds. Training accuracy steadily increases with network size, while validation accuracy is nearly flat and drops for the (128, 64) architecture, indicating mild overfitting.

The chosen configuration balances expressiveness with generalization: a single hidden layer with 128 units, $\alpha = 0.001$, and learning rate 5×10^{-4} . We refer to this as the “tuned” MLP.

4 Validation, Complexity, and Results

4.1 Under- and over-fitting

To visualize under- and over-fitting explicitly, we examined decision trees with different maximum depths and MLPs with different hidden sizes.

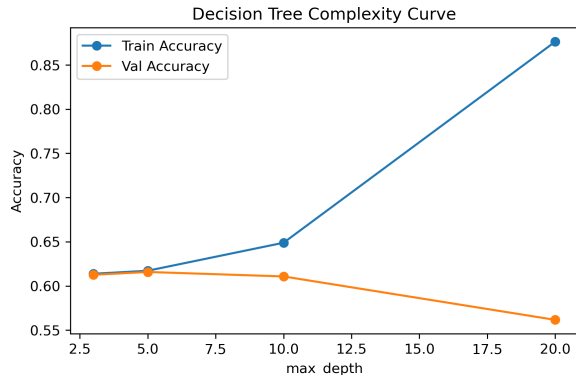


Figure 5: Decision tree complexity curve: deep trees overfit (train accuracy rises, validation accuracy falls).

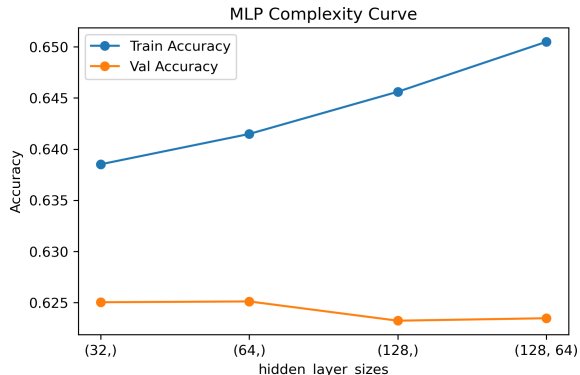


Figure 6: MLP complexity curve: larger networks give only small gains before slightly over-fitting.

For trees, validation accuracy peaked at depth around 5–10 and decreased thereafter, which is typical high-variance behavior. This motivated using Random Forests with moderate depth and a nontrivial `min_samples_leaf`. For the MLP, the gap between training and validation accuracy remained small, but the (128, 64) network began to lose validation performance, suggesting that a single hidden layer is sufficient.

4.2 Test-set performance

All models were trained on the balanced training set (with scaling where appropriate) and evaluated on the original test set of 19,868 encounters. Table 1 summarizes performance.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (balanced)	0.63	0.62	0.55	0.58	0.67
Random Forest (fixed)	0.64	0.62	0.60	0.61	0.69
MLP (fixed 128-64)	0.63	0.61	0.60	0.60	0.68
SVM (RBF)	0.62	0.60	0.60	0.60	0.67
Random Forest (tuned)	0.64	0.62	0.61	0.62	0.69
MLP (tuned 128)	0.63	0.60	0.63	0.61	0.68

Table 1: Test-set performance. Values are rounded to two decimals.

Figure 7 provides a visual comparison of the same metrics.

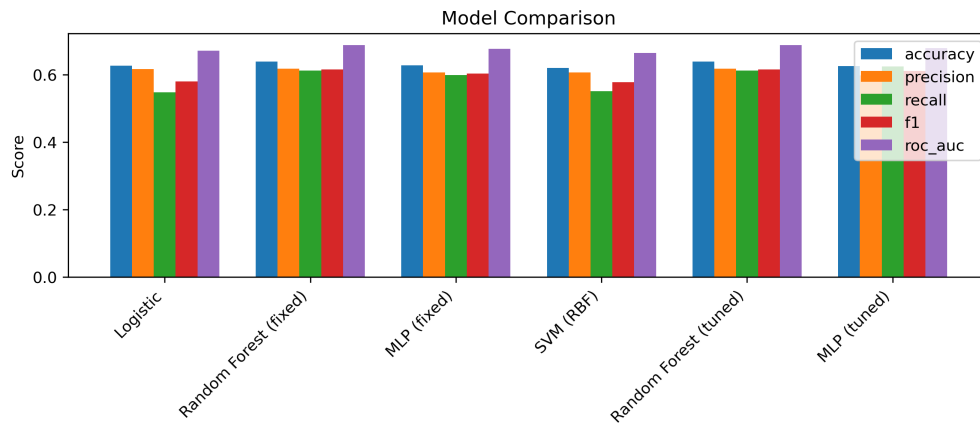


Figure 7: Model comparison across accuracy, precision, recall, F1, and ROC-AUC.

The tuned Random Forest achieves the best overall discrimination (ROC-AUC ≈ 0.69) and a slightly higher F1 score than the other models. The tuned MLP attains the highest recall, which may be desirable when missing high-risk patients is particularly costly. Logistic Regression and SVM perform similarly, indicating that while the relationship between predictors and readmission is somewhat nonlinear, the benefit of complex kernels is limited by noise and feature quality.

Figure 8 shows ROC curves for the main candidate models.

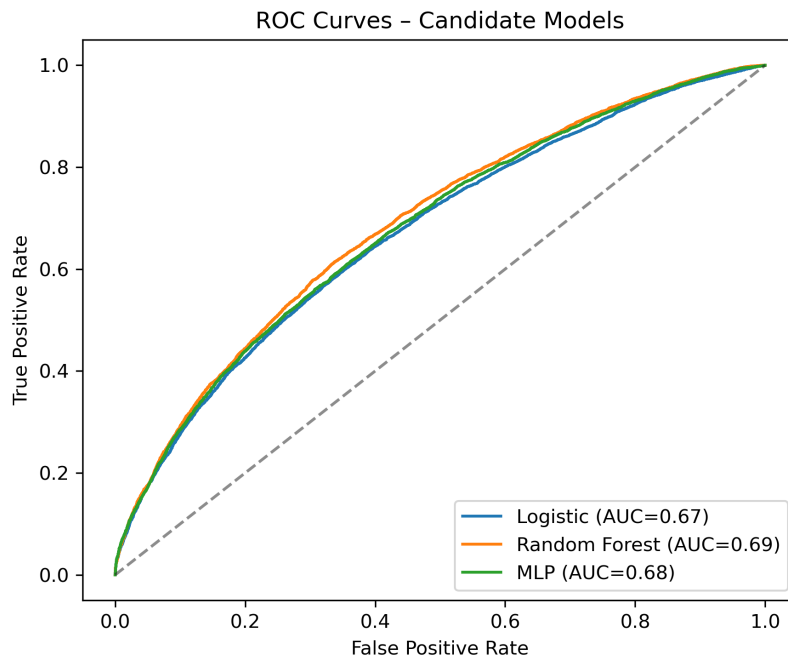


Figure 8: ROC curves for Logistic Regression, tuned Random Forest, and tuned MLP on the test set.

All models outperform random guessing, but the curves illustrate only modest separation, which is typical for clinical prediction tasks based on administrative data.

5 Discussion and Conclusion

This project illustrates a full machine learning workflow on a real clinical dataset, with an emphasis on how model and parameter choices are guided by the data and validation results.

Exploratory analysis revealed heavy missingness in some variables, motivating their removal, and highlighted the importance of utilization measures and diagnosis groups. Feature engineering focused on grouping ICD-9 codes, binarizing medication use, and encoding hospital identifiers. Balancing the training set by downsampling the majority class, while leaving the test set unchanged, produced models with similar precision and recall for the positive class.

Random Forests and MLPs offered the best performance, with Random Forests slightly ahead in ROC-AUC and F1, and MLPs slightly ahead in recall. Complexity curves showed classic under- and over-fitting patterns: deep single trees overfit badly, whereas moderate-depth ensembles and modestly sized neural networks generalize better. SVMs with RBF kernel were competitive but did not provide clear gains over simpler models, possibly due to computational constraints and noisy features.

Several lessons emerge:

- Dataset structure should guide preprocessing; clever feature design is at least as important as sophisticated modeling.
- Hyperparameter tuning is best done iteratively with cross-validation and diagnostic plots, rather than blindly via large grids.

- Evaluating both discrimination (ROC–AUC) and error trade-offs (precision, recall, F1) is essential, especially when false negatives have clinical cost.

Future work could incorporate cost-sensitive learning, probability calibration, or temporal models that exploit multiple encounters per patient. Even modest improvements in prediction could help hospitals better allocate follow-up resources and reduce avoidable readmissions.

Team Contributions

Nathan implemented the preprocessing functions, logistic regression and MLP models, and generated the ROC and complexity plots. Alexander focused on exploratory analysis, feature design (diagnosis and medication encoding), and Random Forest tuning and interpretation. Nhan implemented the SVM experiments, handled train/test splitting and balancing utilities, and helped with evaluation code and figure generation. All members contributed to interpreting results and writing the report.